CLASE 3 - Sistemas Digitales

Microcontroladores: Puertos, GPIO, programación y temporizadores

1. Puertos y registros en AVR

Un puerto agrupa varios pines de entrada y salida. En el AVR ATmega328P, los puertos B, C y D tienen hasta 8 bits asociados, aunque no todos sus pines están disponibles físicamente en todos los encapsulados.

Cada bit de un registro controla o representa el estado de un pin del puerto.

Registros principales del puerto B

Registro	Función
`DDRB`	Define si cada pin funciona como entrada o salida.
`PORTB`	Escribe el valor de salida o activa la resistencia pull-up si el pin es entrada.
`PINB`	Permite leer el estado lógico de los pines.

Por ejemplo, si se configura el bit 1 de `DDRB` en `1`, el pin correspondiente del puerto B funciona como salida. Si se coloca en `0`, funciona como entrada.

En AVR, estos registros siguen una lógica similar para los diferentes puertos:

- `DDR`: selecciona si el pin será entrada o salida.
- `PORT`: establece el valor de salida o activa la resistencia pull-up.
- `PIN`: permite leer el estado lógico del pin.

Resistencia pull-up

La resistencia pull-up mantiene un pin de entrada en nivel lógico alto cuando no está conectado a una señal que lo lleve a nivel bajo.

Por ejemplo, cuando se conecta un botón entre el pin y GND:

Estado del botón	Nivel lógico	Lectura
Sin pulsar	Alto	`1`
Pulsado	Bajo	`0`

En AVR se activa colocando en `1` el bit correspondiente de `PORTB`, siempre que el pin esté configurado como entrada.

Alta impedancia (Hi-Z)

Es un estado en el que el pin presenta una impedancia muy alta y queda prácticamente desconectado eléctricamente del circuito interno de salida.

- Se utiliza cuando el pin funciona como entrada.
- Permite que una señal externa determine su nivel lógico.
- Si no tiene una señal definida ni una resistencia pull-up o pull-down, puede quedar flotante y producir lecturas inestables.

2. Lectura y configuración de pines

Los periféricos del microcontrolador se controlan mediante registros que tienen direcciones dentro del espacio de memoria o de registros de entrada/salida.

Operaciones con registros

Conversión hexadecimal a binario

El valor `0x81` representa:

```
Hexadecimal: 0x81
Binario:     10000001
```

Los bits 7 y 0 están en `1`; los demás están en `0`.

Activar el bit 4 de `PORTB`

```
PORTB |= (1 << 4);
```

La expresión `1 << 4` desplaza el número 1 cuatro posiciones a la izquierda. Luego, el operador OR (`|`) activa el bit 4 sin modificar los demás bits.

Leer el pin D2 con un botón conectado a GND

```
DDRD &= ~(1 << 2);
PORTD |= (1 << 2);

if (PIND & (1 << 2)) {
    // El botón no está pulsado: nivel alto
} else {
    // El botón está pulsado: nivel bajo
}
```

- `DDRD &= ~(1 << 2)`: configura D2 como entrada.
- `PORTD |= (1 << 2)`: activa la resistencia pull-up.
- `PIND`: permite leer el estado real del pin.

Como el botón está conectado a GND, al pulsarlo el pin pasa a nivel bajo (`0`). Por tanto, la condición del `if` será falsa y se ejecutará el `else`.

3. Periféricos, registros y pines

Un periférico es un circuito interno del microcontrolador que cumple una función específica, como controlar entradas, salidas, temporizadores o comunicaciones.

Un pin es una conexión física que permite comunicar el microcontrolador con componentes externos. Puede funcionar como entrada o salida y se controla mediante registros.

Los registros son espacios de almacenamiento internos que permiten controlar o consultar el funcionamiento del microcontrolador. Están implementados mediante circuitos digitales, generalmente construidos a partir de flip-flops.

Al escribir determinados bits en los registros correspondientes, se modifica el comportamiento de los periféricos y de los pines.

En resumen:

- El pin es una conexión física.
- Los registros permiten configurar y controlar su funcionamiento.
- No se puede escribir en cualquier dirección: cada registro tiene una función específica.

4. Estructura de la placa ESP32 DevKit V1

La placa DevKit V1 incorpora un módulo ESP32-WROOM-32 y circuitos adicionales que permiten alimentarlo y programarlo desde una computadora.

Componentes principales

Conexión USB

Proporciona alimentación de 5 V y permite comunicarse con la computadora.

Conversor USB a serial

Chips como el CP2102 o CH340 convierten la comunicación USB de la computadora en comunicación serial que puede utilizar el ESP32.

Regulador de voltaje

Convierte los 5 V provenientes del USB a 3.3 V para alimentar el módulo ESP32.

Módulo ESP32-WROOM-32

Contiene los siguientes componentes:

- **Chip ESP32:** procesador de dos núcleos en la versión habitual del WROOM-32.
- **Memoria Flash:** almacena el programa y otros datos. En la versión común del módulo se encuentran 4 MB.
- **Antena:** permite la comunicación inalámbrica mediante Wi-Fi y Bluetooth.

Los pines GPIO 6 al 11 están asociados normalmente a la comunicación con la memoria Flash del módulo. Por eso, no deben utilizarse como GPIO de propósito general.

Botones de la placa

- `EN`: permite reiniciar el ESP32.
- `BOOT`: permite entrar en el modo de descarga de programas. Está relacionado con GPIO 0.

La placa DevKit V1 puede encontrarse en versiones con diferentes cantidades de pines, por ejemplo, 30 pines.

5. Pines y GPIO del ESP32

GPIO significa *General Purpose Input/Output* (entrada/salida de propósito general). Permite configurar los pines del microcontrolador como entradas o salidas.

Los GPIO permiten:

- Colocar un pin en nivel lógico `0`.
- Colocar un pin en nivel lógico `1`.
- Leer el estado lógico de un pin.

En el ESP32, los niveles lógicos trabajan normalmente con 0 V para el nivel bajo y 3.3 V para el nivel alto. Sus GPIO no deben recibir directamente señales de 5 V.

Tipos de pines

No todos los pines de la placa tienen la misma función. Algunos pueden utilizarse como GPIO, mientras que otros están destinados a alimentación, comunicación o configuración de arranque.

Pines de uso general

Pueden utilizarse para conectar componentes externos, como:

- LEDs.
- Pulsadores.
- Sensores.
- Otros circuitos electrónicos.

Pines de configuración de arranque (Strapping Pins)

Son pines cuyo estado lógico se lee cuando el ESP32 se enciende o reinicia. Según esos niveles, el chip selecciona determinadas opciones de funcionamiento.

Por ejemplo:

- **GPIO0:** si está en nivel bajo durante el arranque, permite entrar en el modo de descarga del programa. Se relaciona con el botón BOOT.
- **GPIO2:** participa en la configuración de arranque y debe respetar las condiciones indicadas para el modo de descarga.
- **GPIO12:** selecciona el voltaje de alimentación de la memoria Flash. Si está en alto durante el arranque, puede seleccionar 1.8 V; en bajo selecciona 3.3 V en la configuración habitual del ESP32 clásico.

Por eso, no se deben conectar componentes que fuercen niveles incorrectos en estos pines durante el arranque.

Pines de solo entrada

Algunos GPIO del ESP32 clásico, como GPIO34, GPIO35, GPIO36 y GPIO39, funcionan únicamente como entradas.

Sus características son:

- Permiten leer señales externas.
- No pueden utilizarse como salidas.
- No disponen de resistencias internas pull-up o pull-down.

Pines de comunicación serial

La comunicación serial permite transmitir información entre el ESP32 y otros dispositivos mediante señales eléctricas.

Utiliza dos líneas principales:

- `TX` (*Transmit*): transmite información.
- `RX` (*Receive*): recibe información.

En una conexión entre dos dispositivos, normalmente TX de uno se conecta a RX del otro.

También se utiliza la comunicación serial para cargar programas y observar mensajes de diagnóstico desde el monitor serie.

Pines de alimentación y reinicio

Estos pines no son GPIO de propósito general.

Pin	Función
`GND`	Tierra o referencia de 0 V del circuito.
`3V3`	Alimentación de 3.3 V. Puede proporcionar energía a circuitos externos dentro de los límites permitidos.
`VIN`	Entrada de alimentación de la placa. En muchas DevKit V1 se utiliza para recibir 5 V, aunque depende del diseño.
`EN`	Permite habilitar o reiniciar el ESP32.

No se debe confundir VIN con una salida fija de 2.5 V. En una DevKit V1 habitual, VIN está asociado a la alimentación de 5 V. Siempre hay que comprobar el diseño de la placa antes de conectar una fuente externa.

Configuración de un GPIO

Para configurar un pin se deben considerar tres aspectos:

1. **Elegir su función:** seleccionar si será entrada, salida o una función alternativa. Esto se relaciona con el multiplexor de funciones (MUX).
2. **Conectar la señal:** establecer la ruta de la señal mediante la matriz de interconexión GPIO.
3. **Habilitar la salida:** configurar el pin para que pueda generar una señal cuando se necesite.

En Arduino, la función `pinMode()` realiza la configuración necesaria para establecer un pin como entrada o salida, sin tener que configurar manualmente cada uno de esos elementos.

```
pinMode(2, OUTPUT);  // Configura GPIO 2 como salida
pinMode(4, INPUT);   // Configura GPIO 4 como entrada
```

Registros GPIO del ESP32

El ESP32 dispone de registros que permiten modificar directamente el estado de sus salidas GPIO.

- **W1TS (Write One to Set):** escribir `1` activa el bit correspondiente; escribir `0` lo mantiene sin cambios.
- **W1TC (Write One to Clear):** escribir `1` desactiva el bit correspondiente; escribir `0` lo mantiene sin cambios.
- **GPIO_IN:** permite leer el estado lógico de los pines de entrada.
- **GPIO_OUT:** representa el estado de salida de los GPIO.

Estos registros permiten controlar los pines mediante operaciones sobre bits, sin modificar innecesariamente los demás.

Riesgo de leer, modificar y escribir

Una operación de lectura, modificación y escritura consiste en:

1. **Leer:** obtener el valor actual del registro.
2. **Modificar:** cambiar el bit o los bits necesarios.
3. **Escribir:** guardar el nuevo valor en el registro.

El problema aparece cuando otra operación, por ejemplo una interrupción, modifica el mismo registro entre la lectura y la escritura. Esto puede provocar que se pierda una modificación o que el registro termine con un valor incorrecto.

Por eso, los registros W1TS y W1TC del ESP32 son útiles: permiten establecer o limpiar bits directamente, sin realizar una operación completa de lectura y modificación.

6. ESP-IDF y programación del ESP32

ESP-IDF (*Espressif IoT Development Framework*) es el entorno de desarrollo oficial de Espressif para programar los microcontroladores ESP32.

Incluye:

- Herramientas para compilar, cargar y depurar programas.
- Bibliotecas y componentes para utilizar los periféricos del ESP32.
- Funciones para controlar GPIO, comunicaciones, Wi-Fi, Bluetooth y otros recursos.

Programa de computadora y programa embebido

Existen diferencias entre un programa que se ejecuta en una computadora y uno que funciona dentro de un microcontrolador.

Programa de computadora	**Programa embebido**
Es iniciado por el sistema operativo.	Comienza a ejecutarse cuando el dispositivo recibe alimentación y completa el arranque.
El sistema operativo administra su ejecución.	Controla directamente el hardware mediante los recursos del microcontrolador y su entorno de ejecución.
Normalmente realiza una tarea y termina.	Generalmente permanece funcionando continuamente.
Cuando termina, devuelve el control al sistema operativo.	Puede seguir ejecutándose mientras el dispositivo tenga alimentación.

En un microcontrolador, el programa normalmente no tiene un final como el de una aplicación de computadora. Permanece ejecutándose para supervisar entradas, controlar salidas y responder a eventos.

Por eso, se puede comparar con un guardia que permanece atento y realiza diferentes tareas durante todo su turno.

Secuencia de arranque y ejecución del ESP32

Cuando se conecta la alimentación al ESP32, el microcontrolador sigue una secuencia antes de comenzar a ejecutar el programa del usuario.

ROM Bootloader

Es el código de arranque que viene integrado de fábrica en el chip y se encuentra almacenado en la memoria ROM.

Sus funciones principales son:

- Ejecutar las instrucciones iniciales de arranque.
- Leer los pines de configuración (*strapping pins*).
- Determinar el modo de arranque.
- Iniciar el proceso de carga del siguiente programa.

Bootloader de Flash

Después de ejecutar el código de la ROM, se carga el segundo nivel del bootloader, almacenado en la memoria Flash.

Sus funciones incluyen:

- Revisar la información de las particiones de la memoria Flash.
- Localizar la aplicación que debe ejecutarse.
- Preparar el proceso para iniciar el programa principal.

Aplicación principal: `app_main()`

En ESP-IDF, la función principal de la aplicación se llama `app_main()`.

```
void app_main(void) {
    // Código principal de la aplicación
}
```

Esta función se ejecuta después del proceso de arranque y permite iniciar las tareas del programa.

FreeRTOS y las tareas

FreeRTOS es un sistema operativo en tiempo real que permite ejecutar y organizar diferentes tareas dentro del microcontrolador.

Una tarea puede encargarse de:

- Leer sensores.
- Controlar LEDs.
- Recibir información por comunicación serial.
- Procesar datos.

En el ESP32 clásico, que tiene dos núcleos, las tareas pueden distribuirse entre ellos. La asignación depende de la configuración del sistema.

`setup()` y `loop()` en Arduino

Cuando se programa el ESP32 utilizando el entorno Arduino, se utilizan dos funciones principales.

`setup()`

Se ejecuta una sola vez al iniciar el programa. Se utiliza para configurar los pines, inicializar la comunicación serial, configurar sensores y preparar las variables y recursos necesarios.

```
void setup() {
    Serial.begin(115200);
    pinMode(2, OUTPUT);
}
```

`loop()`

Se ejecuta repetidamente mientras el microcontrolador permanece funcionando. Se utiliza para leer entradas, procesar información y controlar las salidas.

```
void loop() {
    digitalWrite(2, HIGH);
    delay(1000);

    digitalWrite(2, LOW);
    delay(1000);
}
```

La función `loop()` vuelve a ejecutarse después de terminar cada ciclo. Por eso, normalmente no se utiliza para finalizar el programa, sino para mantener su funcionamiento.

Relación entre ESP-IDF y Arduino: ESP-IDF es el framework oficial de Espressif. Arduino-ESP32 es otro entorno de programación construido sobre ESP-IDF. En Arduino se trabaja principalmente con `setup()` y `loop()`, mientras que en ESP-IDF se utiliza `app_main()` y se pueden crear tareas de FreeRTOS directamente.

Sketch y funciones de programación

Un *sketch* es el nombre que recibe el archivo o programa desarrollado en el entorno Arduino.

Comunicación serial: `Serial.begin()`

Permite iniciar la comunicación serial y establecer la velocidad de transmisión de datos.

```
Serial.begin(115200);
```

El valor `115200` representa la velocidad de transmisión en baudios. El monitor serial debe configurarse con la misma velocidad para mostrar correctamente los mensajes.

Variables estáticas: `static`

La palabra clave `static` permite conservar el valor de una variable durante toda la ejecución del programa. Cuando se declara dentro de una función, la variable conserva su valor entre llamadas a esa función.

```
void loop() {
    static int contador = 0;

    contador++;
    Serial.println(contador);
    delay(1000);
}
```

Aunque `loop()` se ejecute repetidamente, `contador` no vuelve a cero en cada ejecución.

7. Tipos de datos y desbordamiento (Overflow)

En programación de microcontroladores se utilizan tipos de datos que permiten almacenar números dentro de un rango determinado.

Dos tipos comunes son `uint8_t` e `int8_t`.

Tipo de dato	Significado	Rango	Tamaño
`uint8_t`	Entero de 8 bits sin signo.	0 a 255	1 byte
`int8_t`	Entero de 8 bits con signo.	-128 a 127	1 byte

La diferencia está en cómo se interpreta el bit más significativo:

- `uint8_t`: todos los bits se utilizan para representar valores positivos y cero.
- `int8_t`: utiliza una representación con signo, por lo que también permite almacenar valores negativos.

Ejemplo:

```
uint8_t edad = 25;
int8_t temperatura = -10;
```

Ambas variables ocupan un byte, pero permiten almacenar diferentes rangos de valores.

Desbordamiento de variables

El desbordamiento ocurre cuando una variable intenta almacenar un valor que supera el rango permitido por su tipo de dato.

En una variable de 8 bits sin signo (`uint8_t`), el rango es de 0 a 255. Como tiene 256 valores posibles, al superar el máximo el resultado vuelve al inicio del rango.

Por ejemplo:

```
uint8_t numero = 255;
numero++;
```

El resultado almacenado será `0` .

Otro ejemplo:

```
uint8_t x = 200;
uint8_t y = 100;

uint8_t resultado = x + y;
```

La suma es:

$$200 + 100 = 300$$

Pero `uint8_t` no puede almacenar 300. Al superar el límite de 255, el resultado vuelve al inicio del rango:

$$300 - 256 = 44$$

Por lo tanto, el resultado almacenado será `44` .

En C/C++, la suma se realiza primero con promoción de enteros y después el resultado se convierte a `uint8_t` al almacenarse.

En una variable `int8_t` , no se debe asumir que superar el rango produce un resultado válido: el desbordamiento de enteros con signo en C/C++ tiene comportamiento indefinido.

Por eso, es importante conocer los límites de cada tipo de dato y evitar operaciones que los excedan.

8. Funciones de tiempo: `delay()`, `millis()` y Deep Sleep

Función `delay()`

La función `delay()` permite detener temporalmente la ejecución de la tarea durante un tiempo determinado.

```
delay(1000);
```

El valor se expresa en milisegundos:

- `delay(1000)`: espera 1 segundo.
- `delay(500)`: espera medio segundo.
- `delay(100)`: espera 100 milisegundos.

Durante una espera de este tipo, el código de la tarea que la ejecuta queda detenido temporalmente. En el ESP32, el planificador de FreeRTOS puede permitir que otras tareas continúen ejecutándose.

Función `millis()`

Devuelve el tiempo transcurrido desde que comenzó el programa, expresado en milisegundos.

Permite comprobar si ha pasado determinado tiempo sin detener completamente la ejecución del código.

```
unsigned long anterior = 0;

void loop() {
    unsigned long actual = millis();

    if (actual - anterior >= 1000) {
        anterior = actual;
        // Ejecutar una acción cada segundo
    }
}
```

Mientras se espera, el programa puede seguir realizando otras operaciones.

Característica	`delay()`	`millis()`
Funcionamiento	Espera durante un tiempo.	Consulta el tiempo transcurrido.
Bloqueo	Bloquea la tarea que lo ejecuta.	No bloquea por sí mismo.

Característica	`delay()`	`millis()`
Uso	Pausas sencillas.	Control de varias acciones temporizadas.

Consumo de energía y Deep Sleep

El modo *Deep Sleep* permite reducir considerablemente el consumo de energía del ESP32 cuando no necesita estar realizando operaciones.

Se utiliza en dispositivos que funcionan con baterías o que deben permanecer en espera durante largos períodos.

Puede ser útil cuando un programa no necesita ejecutar continuamente su ciclo principal, sino que puede dormir y despertarse posteriormente mediante una fuente de activación configurada.

9. Temporizadores de hardware (Hardware Timers)

Un temporizador es un periférico, es decir, un circuito interno del microcontrolador que realiza una función específica y reduce la carga de trabajo de la CPU.

Es un circuito que cuenta pulsos de reloj. Como cada pulso tiene una duración conocida, permite medir intervalos de tiempo con precisión.

La CPU configura el temporizador y este puede continuar contando de manera independiente. Cuando alcanza un valor determinado, puede generar una alarma o interrupción.

Se puede comparar con un reloj independiente que cuenta el tiempo y avisa cuando se cumple una condición. La CPU no necesita contar manualmente cada pulso; configura el temporizador y puede dedicarse a otras tareas.

Contador del temporizador

El contador es un registro interno del temporizador donde se almacena el valor actual de la cuenta.

Su funcionamiento básico es:

1. La CPU configura el temporizador.
2. Se establece el valor inicial y las condiciones de funcionamiento.
3. La CPU habilita el temporizador.
4. El contador comienza a incrementarse automáticamente.
5. Cuando alcanza el valor de alarma, puede generar un evento.
6. La CPU puede detenerlo, reiniciarlo o modificar su configuración.

El incremento de uno significa que el contador suma una unidad por cada *tick* o pulso de reloj que recibe.

```
0 → 1 → 2 → 3 → 4 → 5 → ...
```

La CPU no necesita ejecutar una instrucción para incrementar cada número: el circuito del temporizador realiza el conteo.

10. Reloj del temporizador

Para medir el tiempo, el temporizador necesita una señal de reloj.

Reloj

Es una señal eléctrica periódica que cambia entre dos niveles:

- Nivel bajo (0).
- Nivel alto (1).

Cada ciclo completo, formado por una subida y una bajada de la señal, representa un período de reloj.

La duración de ese período se calcula mediante:

$$T_s = \frac{1}{F}$$

Donde:

- T_s: período de la señal, en segundos.
- F: frecuencia, en hercios (Hz).

Reloj APB

APB significa *Advanced Peripheral Bus*. Es un bus utilizado para conectar y comunicar la CPU con determinados periféricos del ESP32.

En el ESP32 clásico, el reloj APB normalmente funciona a 80 MHz y sirve como fuente de reloj para los temporizadores tradicionales.

Esto significa que puede proporcionar:

$$80\,\mathrm{MHz} = 80\,000\,000\,\mathrm{Hz}$$

Es decir, 80 millones de ciclos por segundo.

Duración de cada pulso

Si el reloj trabaja a 80 MHz:

$$T_s = \frac{1}{80\,000\,000}$$

$$T_s = 0.0000000125\,\text{s}$$

Equivale a:

$$T_s = 12.5\,\text{ns}$$

Por tanto, cada ciclo de reloj dura 12.5 nanosegundos.

11. Prescaler y resolución del temporizador

Prescaler (divisor)

El prescaler es un divisor de frecuencia que permite reducir la cantidad de pulsos que recibe el contador del temporizador.

En lugar de contar los 80 millones de pulsos por segundo directamente, se divide la frecuencia para trabajar con una más cómoda.

La relación es:

$$F_{\text{timer}} = \frac{F_{\text{reloj}}}{N}$$

Donde:

- F_{timer}: frecuencia con la que incrementa el contador.
- F_{reloj}: frecuencia de entrada, por ejemplo 80 MHz.
- N: divisor o prescaler.

Ejemplo: si el reloj original tiene una frecuencia de 80 MHz y se utiliza un divisor de 80:

$$F_{\text{timer}} = \frac{80\,000\,000}{80}$$

$$F_{\text{timer}} = 1\,000\,000\,\text{Hz}$$

El contador recibe un millón de ticks por segundo. Esto permite que cada incremento represente un microsegundo.

En las gráficas de reloj:

- Una frecuencia alta significa que hay muchos ciclos en poco tiempo.
- Una frecuencia baja significa que hay menos ciclos en el mismo intervalo.
- El prescaler divide la frecuencia, por lo que el contador recibe menos pulsos.

Por ejemplo, si se divide entre 3, el contador recibe un tick por cada tres ciclos de la señal original.

Resolución del temporizador

La resolución es la duración de un tick del contador. Representa el tiempo más pequeño que se puede medir con ese incremento.

Se calcula mediante:

$$R = \frac{1}{F_{\text{timer}}}$$

Donde F_{timer} es la frecuencia que llega al contador después de aplicar el prescaler.

Por ejemplo, si la frecuencia del temporizador es 1 MHz:

$$R = \frac{1}{1\,000\,000}$$

$$R = 0.000001\,\text{s} = 1\,\mu s$$

Cada tick representa un microsegundo.

Relación entre reloj, prescaler y resolución:

1. El reloj proporciona los pulsos.
2. El prescaler divide la frecuencia.
3. El contador recibe los ticks.
4. La resolución indica cuánto tiempo representa cada tick.

12. Alarma, autorrecarga y desbordamiento del contador

Alarma y cálculo del tiempo

La alarma es un valor que se configura en el temporizador para determinar cuándo debe generarse un evento.

El contador avanza hasta alcanzar ese valor. Cuando llega a la alarma, puede generar una interrupción o activar una acción configurada.

Se puede entender mediante esta comparación:

- **Resolución:** duración de cada peldaño.
- **Alarma:** cantidad de peldaños que debe recorrer el contador.
- **Tiempo:** duración total del recorrido.

La fórmula principal es:

$$T = \text{Alarma} \times R$$

También se puede calcular la cantidad de ticks necesarios:

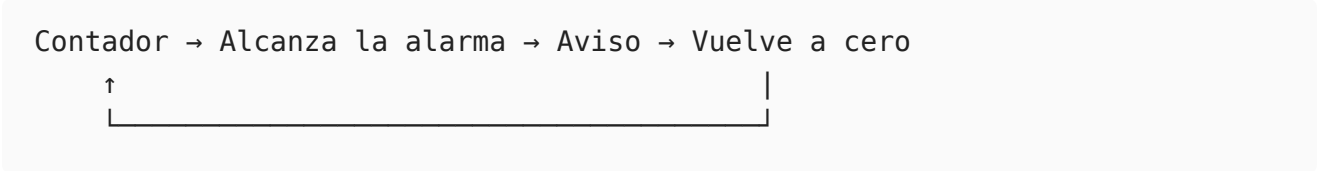
$$\text{Alarma} = T \times F_{\text{timer}}$$

Autorrecarga (Auto-reload)

La autorrecarga permite que el contador vuelva automáticamente a su valor inicial cuando alcanza el valor de alarma.

Su funcionamiento es:

1. El contador comienza a incrementarse.
2. Alcanza el valor de alarma.
3. Se genera el aviso o evento configurado.
4. El contador vuelve a cero.
5. Comienza nuevamente el conteo.



Esto permite generar eventos periódicos sin tener que iniciar manualmente el temporizador después de cada alarma.

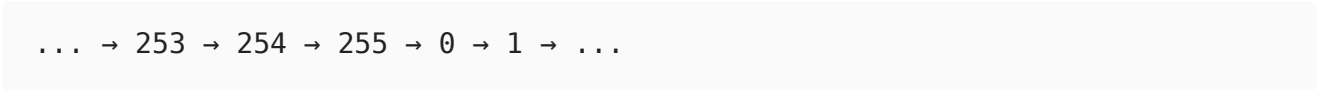
Desbordamiento del contador

El desbordamiento ocurre cuando el contador alcanza su valor máximo y vuelve a cero, debido a que no puede representar un número mayor.

A diferencia de la alarma, el desbordamiento depende del límite de bits del contador.

- La alarma se configura para generar un evento en un valor elegido.
- El desbordamiento ocurre cuando se supera la capacidad máxima del contador.
- Puede aprovecharse intencionalmente para realizar conteos cíclicos.

Por ejemplo, en un contador de 8 bits:



El contador vuelve a cero al superar 255, aunque no haya alcanzado el valor de alarma configurado.

La capacidad máxima de un contador depende de la cantidad de bits que utiliza. Para un contador de n bits sin signo:

$$\text{Máximo} = 2^n - 1$$

Ejemplos:

Bits	Valor máximo
8 bits	255
16 bits	65 535
64 bits	18 446 744 073 709 551 615

El ESP32 clásico utiliza temporizadores generales con contadores de 64 bits. Por eso, su capacidad de conteo es muy grande y el desbordamiento tarda mucho en producirse, dependiendo de la frecuencia de los ticks.

13. Ejercicios de temporizadores

Ejercicio 1

Datos:

- Frecuencia del reloj: 80 MHz.
- Frecuencia deseada del temporizador: 1 MHz.
- Tiempo deseado: 0.5 segundos.

Paso 1. Calcular la resolución

$$R = \frac{1}{1\,000\,000}$$

$$R = 0.000001\,\text{s} = 1\,\mu s$$

Cada tick representa un microsegundo.

Paso 2. Calcular la alarma

$$A = T \times F_{\text{timer}}$$

$$A = 0.5 \times 1\,000\,000$$

$$A = 500\,000\,\text{ticks}$$

Paso 3. Comprobar el tiempo

$$T = A \times R$$

$$T = 500\,000 \times 0.000001$$

$$T = 0.5\,\text{s}$$

Paso 4. Calcular el prescaler

$$N = \frac{F_{\text{reloj}}}{F_{\text{timer}}}$$

$$N = \frac{80\,000\,000}{1\,000\,000}$$

$$N = 80$$

Resultado: se utiliza un divisor de 80, una resolución de 1 microsegundo y una alarma de 500 000 ticks para obtener medio segundo.

Ejercicio 2

Datos:

- Frecuencia deseada del temporizador: 10 000 Hz.
- Alarma: 2 500 ticks.
- Frecuencia del reloj: 80 MHz.

Paso 1. Calcular la resolución

$$R = \frac{1}{10\,000}$$

$$R = 0.0001\,\text{s} = 0.1\,\text{ms}$$

Cada tick representa 0.1 milisegundos.

Paso 2. Calcular el tiempo

$$T = A \times R$$

$$T = 2\,500 \times 0.0001$$

$$T = 0.25\,\text{s}$$

Paso 3. Calcular el prescaler

$$N = \frac{80\,000\,000}{10\,000}$$

$$N = 8\,000$$

Resultado: se necesita un divisor de 8 000, una resolución de 0.1 milisegundos y una alarma de 2 500 ticks para obtener 0.25 segundos.

14. Temporizadores del ESP32 y API

El ESP32 clásico dispone de cuatro temporizadores generales de hardware, organizados en dos grupos:

Grupo	Temporizadores
TIMG0	Timer 0 y Timer 1
TIMG1	Timer 0 y Timer 1

Cada temporizador puede configurarse de forma independiente para contar pulsos, establecer una alarma y generar interrupciones.

Esta organización permite utilizar varios temporizadores para diferentes tareas, por ejemplo:

- Medir intervalos de tiempo.
- Generar eventos periódicos.
- Controlar la frecuencia de ciertas acciones.
- Ejecutar tareas relacionadas con sensores o actuadores.

API para controlar los temporizadores

Una API (*Application Programming Interface*) proporciona funciones ya implementadas que permiten utilizar los recursos del microcontrolador sin tener que configurar manualmente todos sus registros.

En lugar de modificar directamente los registros del temporizador, el programa puede llamar a las funciones de la API para configurarlo.

El proceso general es:

```
Programa
   ↓
Llama a una función de la API
   ↓
La API configura el prescaler y la alarma
   ↓
Se modifican los registros del temporizador
   ↓
Se habilita el contador
```

La API facilita el trabajo porque se encarga de realizar las configuraciones necesarias mediante las funciones disponibles en el framework.

En ESP-IDF existen controladores para los temporizadores. Las funciones concretas dependen de la versión del framework y del periférico utilizado.

15. Interrupciones

Una interrupción es una señal que provoca que el procesador suspenda temporalmente la ejecución normal para atender un evento que necesita una respuesta.

Cuando ocurre una interrupción:

1. El procesador pausa temporalmente el código que estaba ejecutando.
2. Ejecuta una rutina encargada de atender el evento.
3. Una vez atendido, regresa al punto donde había interrumpido la ejecución.

Las interrupciones permiten responder a eventos sin tener que consultar constantemente si algo ocurrió.

Interrupciones internas y externas

Interrupciones internas

Son generadas por eventos dentro del propio microcontrolador o sus periféricos.

Ejemplos:

- Un temporizador alcanza el valor de alarma.
- Se completa una operación de comunicación.
- Se produce un evento de un periférico interno.

Pueden configurarse para ocurrir periódicamente, por ejemplo, cada 500 milisegundos.

Interrupciones externas

Se producen por señales o eventos provenientes del exterior del microcontrolador.

Ejemplos:

- Pulsar un botón.
- Detectar un cambio de nivel lógico en un GPIO.
- Recibir una señal de un sensor externo.

Su momento de ocurrencia puede ser impredecible, porque depende de cuándo sucede el evento externo.

ISR (Interrupt Service Routine)

ISR significa *Interrupt Service Routine* o rutina de servicio de interrupción.

Es la función que se ejecuta cuando ocurre una interrupción. Su propósito es atender rápidamente el evento que la provocó.

Por ejemplo, si un temporizador genera una interrupción, la ISR puede marcar que ha transcurrido un intervalo de tiempo.

Características:

- Se ejecuta como respuesta a una interrupción.
- Interrumpe temporalmente el flujo normal del programa.
- Debe realizar operaciones breves.
- Al terminar, permite continuar con la ejecución normal.

Una ISR no debe realizar operaciones largas ni utilizar funciones bloqueantes como `delay()`. Conviene evitar tareas pesadas y dejar el procesamiento más complejo para el

programa principal o una tarea de FreeRTOS.

Callback

Un callback es una función escrita por el programador que no se llama directamente desde el flujo principal, sino que es invocada por otra función o componente cuando ocurre una condición determinada.

Por ejemplo, al configurar un temporizador, se puede indicar qué función debe ejecutarse cuando se active su alarma.

La relación es:

- El programador escribe la función callback.
- La API recibe o registra esa función.
- Cuando ocurre el evento correspondiente, el sistema la invoca.

En algunos controladores, el callback se ejecuta directamente como parte de una interrupción; en otros, puede ejecutarse desde una tarea. Depende de la API utilizada.

Por eso, es importante saber en qué contexto se ejecuta el callback y qué operaciones se pueden realizar dentro de él.